
Programação Web II

Professor: Leonardo Arruda / Rafael Cruz

Tutorial: API REST com Node.js e MySQL — CRUD Sem ORM

Justificativa

Criar uma API REST com Node.js e MySQL é um passo essencial para quem quer atuar no backend, desenvolver aplicações reais ou integrar sistemas com banco de dados. Trabalhar direto com SQL, sem depender de ORM, ajuda a construir uma base sólida e a entender o que está acontecendo por baixo dos panos.

Neste material você vai construir um CRUD completo (Create, Read, Update, Delete) usando Node.js, Express e MySQL, sem ORM, do jeito que acontece em muitos projetos reais de backend. Ao final, você terá uma API REST rodando localmente, conectada ao MySQL, pronta para ser consumida por um frontend, app mobile ou qualquer outro cliente.

Objetivos

- Compreender o que é uma API REST e o papel de cada operação do CRUD.
- Estruturar um projeto Node.js de forma organizada, sem ORM.
- Configurar variáveis de ambiente com dotenv.
- Conectar a aplicação ao banco de dados MySQL usando mysql2/promise.
- Implementar as quatro operações do CRUD com Express.
- Testar cada endpoint com Insomnia, Postman ou Thunder Client.
- Refatorar o projeto aplicando o padrão MVC.

Tecnologias Utilizadas

- Node.js — ambiente de execução JavaScript no servidor
- Express — framework para criação de rotas e servidor HTTP
- mysql2/promise — driver de conexão com o MySQL, com suporte a async/await
- MySQL (ou MariaDB) — banco de dados relacional
- dotenv — gerenciamento de variáveis de ambiente
- cors — middleware para comunicação entre domínios diferentes
- MySQL Workbench — criação e administração do banco
- Insomnia / Postman / Thunder Client — testes dos endpoints

Estrutura Inicial do Projeto Node.js

Antes de escrever qualquer código, você precisa de uma estrutura mínima e organizada. A estrutura de pastas e arquivos será assim:

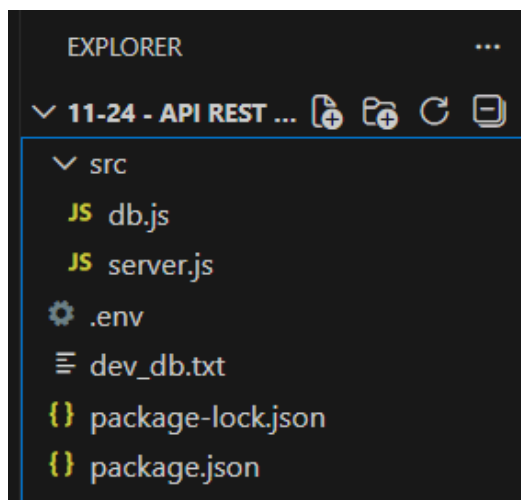


Figura 1 – Estrutura de pastas do projeto

O papel de cada arquivo

Arquivo	Função
server.js	Arquivo principal da API: endpoints, middlewares e inicialização do servidor Express.
db.js	Responsável exclusivamente pela conexão com o MySQL, usando pool de conexões.
.env	Armazena variáveis de ambiente como porta da API e credenciais do banco de dados.
dev_db.txt	Script SQL para criar o banco, a tabela e inserir dados de teste.
package.json	Define as dependências do projeto, scripts e configurações do Node.js.
package-lock.json	Garante que todas as dependências sejam instaladas nas mesmas versões exatas, mantendo o projeto consistente entre máquinas diferentes.

Configurando o Projeto como Módulo ES ("type": "module")

Repare que, mais adiante, o db.js e o server.js usam a sintaxe `import ... from "..."`. Essa é a sintaxe de ES Modules (ESM), e não a sintaxe padrão que o Node.js usa por padrão (CommonJS, com `require`).

Se você tentar rodar um arquivo com `import` sem configurar o projeto corretamente, o Node.js retorna um erro do tipo:

```
SyntaxError: Cannot use import statement outside a module
```

Para que o Node.js interprete os arquivos .js do projeto como módulos ES, é preciso declarar isso no package.json, adicionando o campo `"type": "module"`:

Arquivo: `package.json`

```
{  
  "name": "api-rest-node-mysql",  
  "version": "1.0.0",
```

```
{
  "type": "module",
  "main": "server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mysql2": "^3.11.0"
  }
}
```

Pontos importantes

- Com "type": "module" definido, todos os arquivos .js do projeto passam a ser tratados como ESM. Não é possível misturar require e import no mesmo arquivo.
- Essa configuração também afeta como __dirname e __filename funcionam: eles não existem por padrão em ESM (não são necessários no código deste tutorial, mas vale saber para projetos futuros).
- Alternativa: em vez de configurar "type": "module" no package.json, também é possível nomear os arquivos com a extensão .mjs. Mas para manter o projeto consistente, o mais comum é configurar o package.json uma única vez.
- Essa configuração deve ser feita antes de escrever db.js e server.js, já que ambos dependem da sintaxe import.

Criando o Banco de Dados e a Tabela no MySQL

Antes de conectar o Node.js, você precisa ter o banco pronto. No arquivo dev_db.txt, use o seguinte SQL:

```
1 CREATE DATABASE IF NOT EXISTS dev_db
2 DEFAULT CHARACTER SET utf8mb4
3 DEFAULT COLLATE utf8mb4_unicode_ci;
4
5 USE dev_db;
6
7 CREATE TABLE IF NOT EXISTS usuarios (
8     id INT NOT NULL AUTO_INCREMENT,
9     nome VARCHAR(100) NOT NULL,
10    email VARCHAR(120) NOT NULL UNIQUE,
11    PRIMARY KEY (id)
12 );
13
14 INSERT INTO usuarios (nome, email) VALUES
15 ('Nome 1', 'nome1@dominio.com'),
16 ('Nome 2', 'nome2@dominio.com'),
17 ('Nome 3', 'nome3@dominio.com'),
18 ('Nome 4', 'nome4@dominio.com');
```

Figura 2 – Script de criação do banco e da tabela usuarios

Execute esse script no MySQL Workbench ou no MariaDB (ou qualquer SGBD MySQL disponível em seu computador). Ele cria o banco, a tabela usuarios e já insere alguns registros para teste.

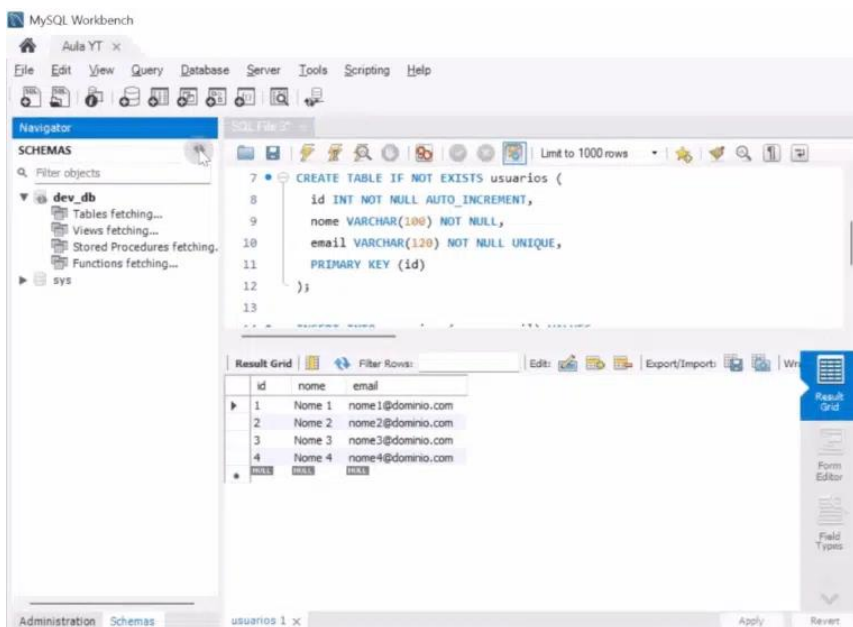


Figura 3 – Resultado da execução do script no MySQL Workbench

Configurando Variáveis de Ambiente com dotenv

Nunca coloque credenciais direto no código. Crie um arquivo .env na raiz do projeto:

Arquivo: .env

```
PORT=3002
DB_HOST=localhost
DB_USER=root
DB_PASSWORD="sua_senha"
DB_DATABASE=dev_db
DB_PORT=3306
```

Essas informações variam de acordo com o seu ambiente. Em produção, esse arquivo não deve subir para o GitHub (adicione .env ao .gitignore).

Conectando o Node.js ao MySQL (criando o arquivo db.js)

Agora você cria a conexão com o banco usando mysql2/promise.



```
1 import mysql from "mysql2/promise";
2 import dotenv from "dotenv";
3
4 dotenv.config();
5
6 export const pool = mysql.createPool({
7   host: process.env.DB_HOST,
8   user: process.env.DB_USER,
9   password: process.env.DB_PASSWORD,
10  database: process.env.DB_DATABASE,
11  port: Number(process.env.DB_PORT),
12  waitForConnections: true,
13  connectionLimit: 10,
14 });
```

Figura 4 – Arquivo db.js: pool de conexões com o MySQL

Aqui você cria um pool de conexões, que melhora desempenho e evita abrir uma nova conexão a cada requisição. Assim você tem uma conexão pronta para ser reutilizada em qualquer parte da API.

Criando a API REST com Express (server.js)

No server.js, começa a estrutura básica da API:



Figura 5 – Configuração inicial do server.js

Esses middlewares permitem:

- Comunicação entre domínios (CORS)
- Receber dados em JSON no corpo das requisições

O Que é CRUD em uma API REST?

CRUD é um acrônimo que representa as quatro operações básicas que uma aplicação faz ao trabalhar com banco de dados:

Create: criar um novo registro

Read: ler ou listar registros

Update: atualizar um registro existente

Delete: remover um registro

Operação	Verbo HTTP
Create	POST
Read	GET
Update	PUT ou PATCH
Delete	DELETE

CREATE – Criar Usuário no Banco de Dados

```
1 app.post("/", async (req, res) => {
2   try {
3     const { nome, email } = req.body;
4
5     const [result] = await pool.query(
6       "INSERT INTO usuarios (nome, email) VALUES (?, ?)",
7       [nome, email]
8     );
9
10    res.status(201).json({ id: result.insertId, nome, email });
11  } catch (e) {
12    res.status(500).json({ erro: "Falha ao criar usuário" });
13  }
14 });
```

Figura 6 – Endpoint POST / (criar usuário)

Boa prática importante aqui: SQL parametrizado (os ? na query), que evita SQL Injection. O usuário é criado no banco e a resposta retorna com status 201 Created.

READ – Listar Usuários

```
1 app.get("/", async (req, res) => {
2   try {
3     const [rows] = await pool.query("SELECT * FROM usuarios");
4     res.json(rows);
5   } catch (e) {
6     res.status(500).json({ erro: "Falha ao listar usuários" });
7   }
8 });
```

Figura 7 – Endpoint GET / (listar usuários)

Esse endpoint retorna todos os registros da tabela usuarios.

UPDATE – Atualizar Usuário

```
1 app.put("/:id", async (req, res) => {
2   try {
3     const { id } = req.params;
4     const { nome, email } = req.body;
5
6     const [result] = await pool.query(
7       "UPDATE usuarios SET nome = COALESCE(?, nome), email = COALESCE(?, email) WHERE id = ?",
8       [nome || null, email || null, id]
9     );
10
11    if (!result.affectedRows) {
12      return res.status(404).json({ erro: "Usuário não encontrado" });
13    }
14
15    res.json({ mensagem: "Atualizado com sucesso" });
16  } catch (e) {
17    res.status(500).json({ erro: "Falha ao atualizar usuário" });
18  }
19 });
```

Figura 8 – Endpoint PUT /:id (atualizar usuário)

O COALESCE garante que apenas os campos enviados na requisição sejam alterados.

DELETE – Remover Usuário

```
1 app.delete("/:id", async (req, res) => {
2   try {
3     const { id } = req.params;
4
5     const [result] = await pool.query(
6       "DELETE FROM usuarios WHERE id = ?",
7       [id]
8     );
9
10    if (!result.affectedRows) {
11      return res.status(404).json({ erro: "Usuário não encontrado" });
12    }
13
14    res.json({ mensagem: "Deletado com sucesso" });
15  } catch (e) {
16    res.status(500).json({ erro: "Falha ao deletar usuário" });
17  }
18 });
```

Figura 9 – Endpoint DELETE /:id (remover usuário)

Rodando o Servidor

Finalize o server.js com:

```
1 app.listen(PORT, () => {  
2   console.log(`Servidor MySQL rodando em http://localhost:${PORT}`);  
3 });
```

Figura 10 – app.listen no server.js

No package.json, use:

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js"  
}
```

E rode no terminal:

```
npm install  
npm run dev
```

Testando a API REST

Ferramentas recomendadas para testar os endpoints:

- Insomnia
- Postman
- Thunder Client

Roteiro de testes

Faça os testes na ordem abaixo. Para cada um, anote na sua ferramenta o status HTTP retornado e o corpo da resposta, e compare com o "resultado esperado".

Teste 1 — Listar usuários (READ)

- Método: GET
- URL: /
- Corpo: nenhum
- Resultado esperado: status 200 e um array JSON com os quatro usuários inseridos pelo dev_db.txt.

Teste 2 — Criar um usuário válido (CREATE)

- Método: POST — URL: / — Corpo (JSON): { "nome": "Ana Souza", "email": "ana.souza@exemplo.com" }
- Resultado esperado: status 201 e um JSON com id, nome e email do usuário recém-criado. Rode o Teste 1 de novo e confirme que o novo usuário aparece na lista.

Teste 3 — Criar um usuário sem o campo email (validação)

- Método: POST — URL: / — Corpo (JSON): { "nome": "Usuário Incompleto" }
- Resultado esperado: como a tabela tem email NOT NULL, o MySQL rejeita o insert e o catch do endpoint devolve status 500 com { "erro": "Falha ao criar usuário" }.
- Pergunta para reflexão: um status 500 genérico é uma boa resposta para esse caso? Que status seria mais adequado (dica: 400 Bad Request) e o que precisaria mudar no código para diferenciar "erro de validação" de "erro interno do servidor"?

Teste 4 — Criar um usuário com e-mail duplicado

- Método: POST — URL: / — Corpo (JSON): repita o e-mail usado no Teste 2, com um nome diferente.
- Resultado esperado: a coluna email é UNIQUE, então o insert falha e a API retorna 500. Mesma pergunta do Teste 3: como você trataria esse caso de forma mais específica (ex.: 409 Conflict)?

Teste 5 — Atualizar um usuário existente (UPDATE)

- Método: PUT — URL: /:id (use o id retornado no Teste 2) — Corpo (JSON): { "nome": "Ana Souza Silva" }
- Resultado esperado: status 200 e { "mensagem": "Atualizado com sucesso" }. Repita o Teste 1 e confirme que só o nome mudou, o email continua o mesmo (efeito do COALESCE).

Teste 6 — Atualizar um usuário que não existe

- Método: PUT — URL: /999999 (um id que não existe na tabela) — Corpo (JSON): { "nome": "Não Importa" }
- Resultado esperado: status 404 e { "erro": "Usuário não encontrado" }.

Teste 7 — Remover um usuário existente (DELETE)

- Método: DELETE — URL: /:id (use o id do usuário criado no Teste 2)
- Resultado esperado: status 200 e { "mensagem": "Deletado com sucesso" }. Repita o Teste 1 e confirme que o usuário sumiu da lista.

Teste 8 — Remover um usuário que não existe

- Método: DELETE — URL: /999999
- Resultado esperado: status 404 e { "erro": "Usuário não encontrado" }.

Entregável do roteiro de testes

Monte uma tabela (print, planilha ou markdown) com uma linha por teste, contendo: número do teste, método, URL, corpo enviado, status recebido e se o resultado bateu com o esperado. Para os Testes 3 e 4, escreva em duas ou três linhas sua resposta para a pergunta de reflexão.

Desafio Final: Aplicando o Padrão MVC

Até aqui, o projeto funciona, mas concentra tudo em dois arquivos: db.js cuida da conexão, e server.js acumula rotas, lógica de negócio e acesso ao banco no mesmo lugar. Isso funciona para um projeto pequeno, mas dificulta manutenção, testes e trabalho em equipe conforme o projeto cresce.

Seu desafio: reorganizar este mesmo projeto (sem mudar o comportamento da API) aplicando o padrão MVC (Model-View-Controller), adaptado para uma API REST (sem View, já que não há HTML sendo renderizado — a "resposta" é o próprio JSON).

Estrutura de pastas esperada

```
projeto/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   └── usuarioModel.js
│   ├── controllers/
│   │   └── usuarioController.js
│   ├── routes/
│   │   └── usuarioRoutes.js
│   └── server.js
├── .env
├── dev_db.txt
├── package.json
└── package-lock.json
```

O que vai em cada camada

Camada	Responsabilidade	O que NÃO deve ter
Model (usuarioModel.js)	Todas as queries SQL relacionadas a usuarios: criar, listar, atualizar, deletar. Cada função recebe dados simples e retorna dados do banco.	Nenhuma referência a req ou res. O model não sabe que existe HTTP.
Controller (usuarioController.js)	Recebe req e res, extrai os dados da requisição, chama o model, decide o status HTTP e monta a resposta.	Nenhuma query SQL escrita diretamente aqui. Quem fala com o banco é o model.
Routes (usuarioRoutes.js)	Define os endpoints (GET /, POST /, PUT /:id, DELETE /:id) e liga cada um a função correspondente do controller.	Nenhuma lógica de negócio nem SQL. Só mapeamento de rota para controller.
Config (config/db.js)	Só a criação do pool de conexões, exatamente como já está hoje.	—
server.js	Cria a instância do Express, registra os middlewares (cors, express.json) e conecta as rotas (app.use(...)). Praticamente nada além disso.	Nenhuma rota nem query direta.

Regras do desafio

- O comportamento da API não pode mudar: os mesmos quatro endpoints, os mesmos status HTTP, as mesmas respostas de antes.
- Todos os testes do Roteiro de testes precisam continuar passando depois da refatoração.
- Nenhuma query SQL pode aparecer fora dos arquivos de models/.
- Nenhum app.get, app.post, app.put ou app.delete pode aparecer fora de routes/.
- Use export/import (o projeto já está configurado como módulo ES) para conectar os arquivos entre si.

Dica para começar

Comece pelo model. Pegue as quatro queries que já existem em server.js (dentro dos blocos pool.query(...)) e transforme cada uma em uma função exportada, por exemplo criarUsuario(nome, email), listarUsuarios(), atualizarUsuario(id, nome, email) e deletarUsuario(id). Só depois de isolar o acesso ao banco, mexa no controller e nas rotas.

Próximos Passos Possíveis

- Autenticação com JWT
- Validação de dados
- Paginação
- Integração com frontend
- Deploy em servidor

FAQ

Vale a pena criar API REST sem ORM?

Sim. Trabalhar sem ORM ajuda a entender SQL de verdade e o funcionamento real do banco, o que é muito valorizado no mercado.

Esse CRUD serve para projetos reais?

Serve como base. Em produção, adicione validações, autenticação e tratamento de erros mais avançado.

Posso usar PostgreSQL no lugar do MySQL?

Pode. A lógica do CRUD é a mesma, mudando apenas o driver e algumas diferenças de SQL.

Preciso saber SQL para usar Node.js no backend?

Não é obrigatório, mas saber SQL amplia muito suas possibilidades e controle do sistema.

Essa API pode ser usada com frontend React ou Vue?

Sim. Qualquer frontend que faça requisições HTTP pode consumir essa API.

Conclusão

Você aprendeu a construir uma API REST com Node.js e MySQL sem utilizar um ORM, escrevendo manualmente as consultas SQL para as quatro operações do CRUD, configurando variáveis de ambiente e testando cada endpoint. Esse conhecimento fortalece a compreensão sobre o funcionamento real da comunicação entre backend e banco de dados, sendo uma base sólida para evoluir posteriormente para o padrão MVC, autenticação, validação de dados e deploy em produção. Continue explorando!

Referências

EXPRESS. Express — Node.js web application framework. Disponível em: <https://expressjs.com/>. Acesso em: 30 ago. 2026.

NODE.JS. Documentação oficial. Disponível em: <https://nodejs.org/en/docs>. Acesso em: 30 ago. 2026.

MySQL2. Pacote mysql2 — documentação. Disponível em: <https://www.npmjs.com/package/mysql2>. Acesso em: 30 ago. 2026.

MYSQL. MySQL — Documentação. Disponível em: <https://dev.mysql.com/doc/refman/8.4/en/>. Acesso em: 30 ago. 2026.

POSTMAN. Postman — Documentação. Disponível em: <https://learning.postman.com/docs/introduction/overview/>. Acesso em: 30 ago. 2026.